

AR and VR Based Industrial Training System (Digital Twin)



Indian Institute Of Information Technology ,Sri City

**Project Report By
K.Sai Madhav
Department Of ECE**

**Under the Guidance Of
Dr.Raja Vara Prasad
Department of ECE,IIIT Sri City**

Date: 26/06/2026

Abstract

- This project presents the design and development of a Digital Twin-based Robotic Arm System for real-time monitoring, simulation, and control. The system consists of a 5-Degree-of-Freedom (5-DOF) robotic arm fabricated using 3D-printed components and actuated by SG90 servo motors. A virtual replica of the robotic arm is created in Unity, enabling real-time visualization and interaction with the physical system.
- The physical robotic arm is controlled using a microcontroller-based architecture integrated with wireless communication for remote operation. To improve the intelligence of the system, a machine learning model is developed using a dataset generated from the robotic arm's forward kinematics. The model predicts joint angles based on the desired end-effector position, enabling efficient inverse kinematic estimation. The generated dataset includes various combinations of joint angles and corresponding Cartesian coordinates of the robotic arm.
- The Digital Twin framework establishes synchronization between the physical and virtual environments, allowing users to observe arm movements, validate control strategies, and analyze system performance in a risk-free virtual space. The project demonstrates the potential of Digital Twin technology in robotics, industrial training, and automation applications by combining physical hardware, simulation, and artificial intelligence techniques.
- The developed system provides a cost-effective platform for robotic experimentation, remote monitoring, operator training, and future research in smart manufacturing and cyber-physical systems.

Introduction

Project Overview

1. Project Description

This project focuses on developing a Digital Twin framework for a 5-DOF robotic arm. The system combines a physical robotic arm fabricated using 3D-printed components with a virtual model created in Unity. The digital twin enables visualization and interaction with the robotic arm in a simulated environment, providing a platform for testing and analysis without affecting the physical hardware.

2. Project Significance

The project demonstrates the application of Digital Twin technology in robotics and automation. By integrating simulation, embedded systems, and machine learning, the system provides a cost-effective solution for operator training, robotic system monitoring, virtual testing, and educational research. The developed platform can be extended for future industrial and smart manufacturing applications.

Motivation

- With the rapid advancement of Industry 4.0 technologies, there is an increasing demand for intelligent systems that can improve efficiency, safety, and productivity in industrial environments. Traditional robotic systems often require extensive physical testing, which can be time-consuming, costly, and prone to operational risks. Digital Twin technology offers a promising solution by creating a virtual representation of a physical system, enabling real-time monitoring, simulation, and analysis.
- The motivation behind this project is to explore the integration of robotics, simulation, and artificial intelligence through the development of a Digital Twin-based robotic arm. By creating a synchronized virtual model of a physical robotic arm, users can visualize operations, test different scenarios, and analyze system behavior without affecting the actual hardware. This approach reduces development time, minimizes potential damage to equipment, and provides a safe environment for experimentation and training.
- Furthermore, the project aims to provide a cost-effective learning and research platform for understanding modern concepts such as Digital Twins, machine learning, embedded systems, and robotic control. The knowledge gained from this project can contribute to future applications in industrial automation, remote monitoring, smart manufacturing, and intelligent robotic systems.

1.1 Background

Industry 4.0

Industry 4.0 represents the fourth industrial revolution, where advanced technologies such as the Internet of Things (IoT), Artificial Intelligence (AI), robotics, cloud computing, and big data are integrated into industrial processes. It enables smart, connected systems that communicate and operate in real time, leading to improved efficiency, productivity, and flexibility in manufacturing and automation.

Smart Manufacturing

Smart Manufacturing is the use of advanced digital technologies to monitor, control, and optimize production processes. It relies on real-time data, automation, and intelligent decision-making to improve product quality, reduce downtime, increase operational efficiency, and enable customized production based on market demands.

Digital Twin Technology

Digital Twin Technology is the creation of a virtual replica of a physical system, process, or product. The digital twin continuously receives data from the physical system and replicates its behavior in a virtual environment. This technology enables real-time monitoring, performance analysis, predictive maintenance, simulation, and better decision-making throughout the system's lifecycle.



1.2 Problem Statement

- Real machine training is costly and risky.
- Damage to equipment affects production and safety.
- Industrial systems require skilled operators.
- Limited access to real industrial environments for practice.
- Beginners need repeated training before real operation.

Example Areas:

- Harbour crane systems
- Robotic automation
- Manufacturing industries
- Construction equipment
- Warehouse automation



1.3 Proposed Solution

- Ar and Vr based Industrial System
1. A realistic virtual industrial environment is developed using Unity.
 2. AI monitors user performance and provides feedback.
 3. Sensors and Arduino boards enable hardware interaction.
 4. Trainees can safely practice industrial operations virtually.
 5. The system supports real-time simulation and learning.

Technologies Used:

- Unity 3D
- AI/ML
- Microcontroller boards
- Sensors
- Simulation & VR

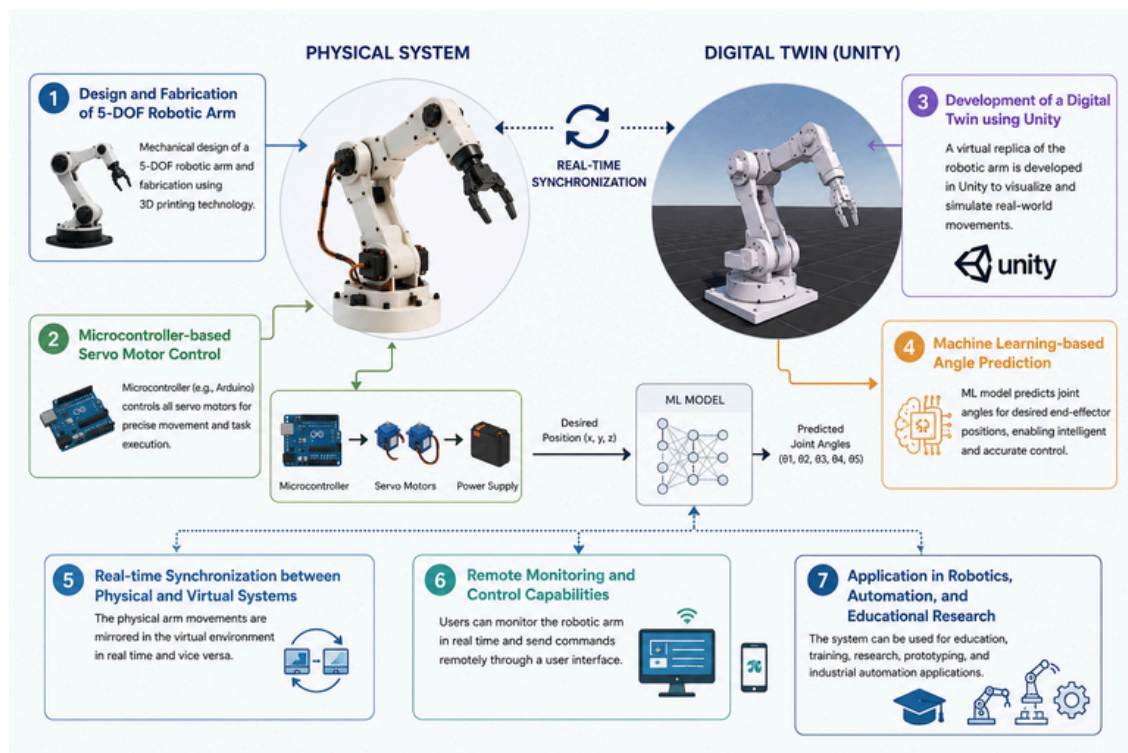


1.4 Objectives

1. Design and develop a 5-DOF robotic arm for object manipulation and experimentation.
2. Fabricate the robotic arm using 3D printing technology to achieve a lightweight and cost-effective structure.
3. Implement microcontroller-based control for servo motor operation and real-time robotic arm movement.
4. Develop a Digital Twin in Unity to create a virtual representation of the physical robotic arm.
5. Implement machine learning-based angle prediction for intelligent robotic arm control and inverse kinematics estimation.
6. Enable real-time monitoring and remote control between the physical system and its digital counterpart.

1.5 Scope Of Work

- Design and fabrication of a 5-DOF robotic arm.
- Microcontroller-based servo motor control.
- Development of a Digital Twin using Unity.
- Machine learning-based angle prediction.
- Real-time synchronization between physical and virtual systems.
- Remote monitoring and control capabilities.
- Application in robotics, automation, and educational research.



Chapter 2: Design and Development of Physical Robotic Arm

2.1 Mechanical Design

The mechanical design of the robotic arm was developed to achieve flexibility, stability, and ease of control while maintaining a lightweight structure. A 5-Degree-of-Freedom (5-DOF) configuration was selected to provide sufficient mobility for performing object manipulation and positioning tasks. The five joints enable the arm to perform base rotation, shoulder movement, elbow movement, wrist movement, and gripper operation.

- **Joint arrangement**

The joints are arranged in a serial manipulator configuration, where each joint contributes to the overall motion of the end effector. SG90 servo motors are used at each joint to provide rotational motion and precise angular control. This arrangement allows the robotic arm to reach different positions within its workspace while maintaining a simple and cost-effective design.

- **Link lengths**

Link	Length
L1	2 cm
L2	6.2 cm
L3	6.3 cm
L4	6.7 cm

The robotic arm consists of four main links with lengths of $L1 = 2\text{ cm}$, $L2 = 6.2\text{ cm}$, $L3 = 6.3\text{ cm}$, and $L4 = 6.7\text{ cm}$. These link dimensions were selected to provide an adequate working range while ensuring structural stability and minimizing the load on the servo motors. The combination of link lengths and joint configuration enables the arm to achieve smooth movement and effective reach for pick-and-place operations and experimental tasks.

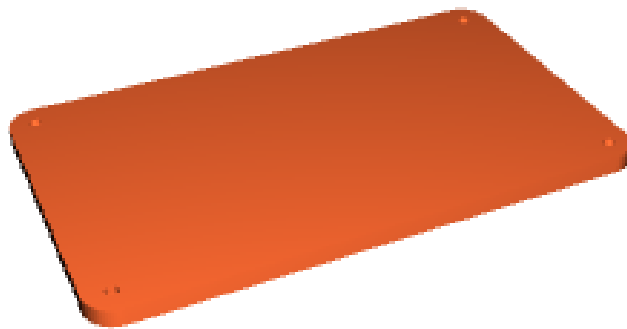
2.2 CAD Modeling

For the development of the robotic arm, pre-designed STL models of the arm components were utilized. The STL files were obtained from existing open-source designs and were reviewed to ensure compatibility with the project requirements.

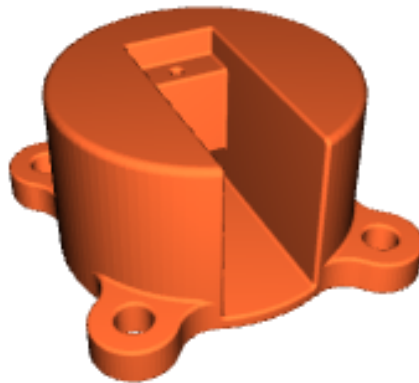
The imported models were used for 3D printing and assembly of the physical robotic arm. These STL models were also incorporated into the simulation environment to develop the Digital Twin, ensuring consistency between the physical and virtual systems.

CAD images

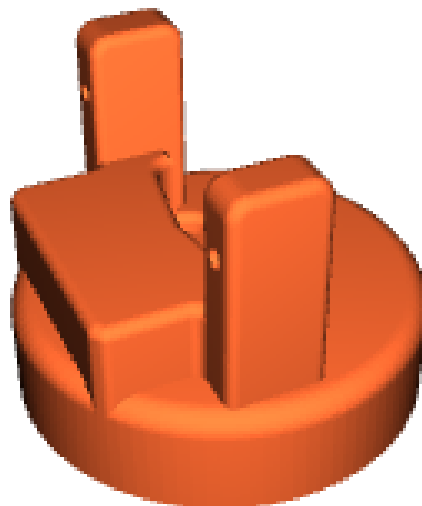
Case_lid



Base1



Base 2



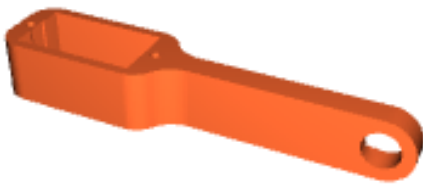
Arm 1



Gear arm1



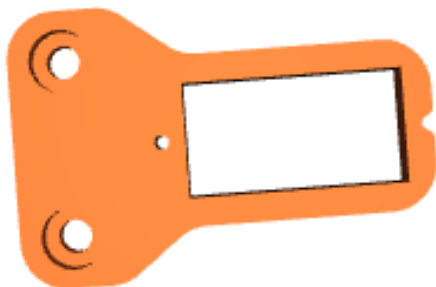
Arm 2



Gear arm2



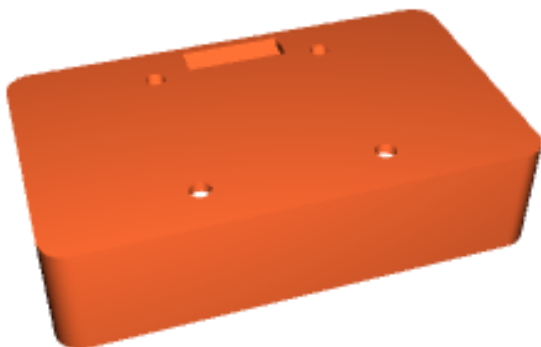
Arm support



Gear servo



Case



Gear



2.3 3D Printing Process

Printer Used: Ender 3 V2

The robotic arm components were fabricated using the Ender 3 V2 3D printer, a widely used FDM (Fused Deposition Modeling) printer known for its reliability and printing accuracy. The printer was utilized to manufacture the arm links, joints, and supporting structures from STL models.

The use of 3D printing enabled rapid prototyping, reduced manufacturing costs, and allowed easy replacement or modification of components. The printed parts were subsequently assembled with servo motors and mechanical fasteners to construct the complete robotic arm.

Material Used: PLA

PLA (Polylactic Acid) filament was used for manufacturing the robotic arm components through 3D printing. PLA was selected due to its ease of printing, good dimensional accuracy, lightweight nature, and cost-effectiveness.

The material provides sufficient strength for the robotic arm's structural components while maintaining a low overall weight, which helps reduce the load on the SG90 servo motors. Additionally, PLA offers good surface finish and is suitable for rapid prototyping and educational robotics applications.

Printing Parameters

Parameter	Value
Layer Height	0.2 mm
Infill	20%
Nozzle Diameter	0.4 mm
Print Speed	50 mm/s

2.4 Fabricated Parts



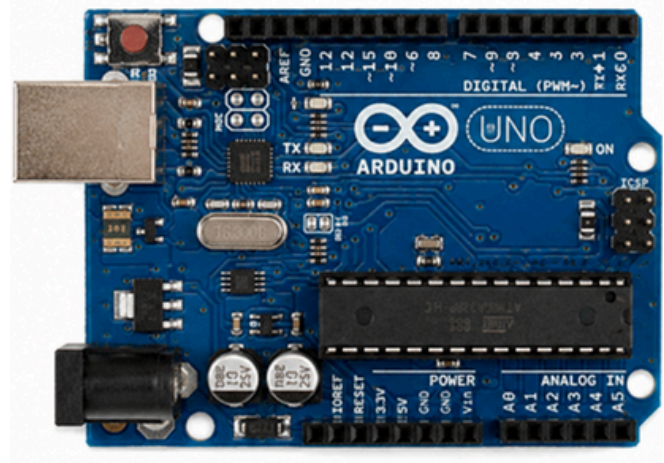
- The fabricated parts of the robotic arm were manufactured using a 3D printer based on the CAD models designed for the project. PLA (Polylactic Acid) filament was selected as the printing material due to its lightweight nature, ease of printing, dimensional stability, and sufficient mechanical strength for prototype development. Each component was printed individually and later assembled to form the complete 5-DOF robotic arm.
- The fabricated components include the base platform, base support frame, servo motor mounts, shoulder link, elbow link, wrist link, gripper mechanism, gripper fingers, gear wheels, and connecting brackets. These parts were designed with appropriate mounting holes to accommodate SG90 servo motors and fastening screws, ensuring accurate alignment and smooth rotational movement of each joint.
- After fabrication, all parts were carefully inspected for dimensional accuracy, surface quality, and proper fit. Minor finishing operations, such as removing support material and smoothing the edges, were carried out before assembly. The fabricated components were then assembled using screws and servo motors to obtain a rigid and lightweight robotic arm capable of performing controlled pick-and-place operations.
- The use of 3D-printed fabricated parts significantly reduced manufacturing cost and development time while allowing easy modification and replacement of individual components. This approach also provided flexibility for future design improvements and supported the rapid prototyping requirements of the digital twin-based robotic arm system.

Chapter 3: Hardware and Software Architecture

3.1 Hardware Components

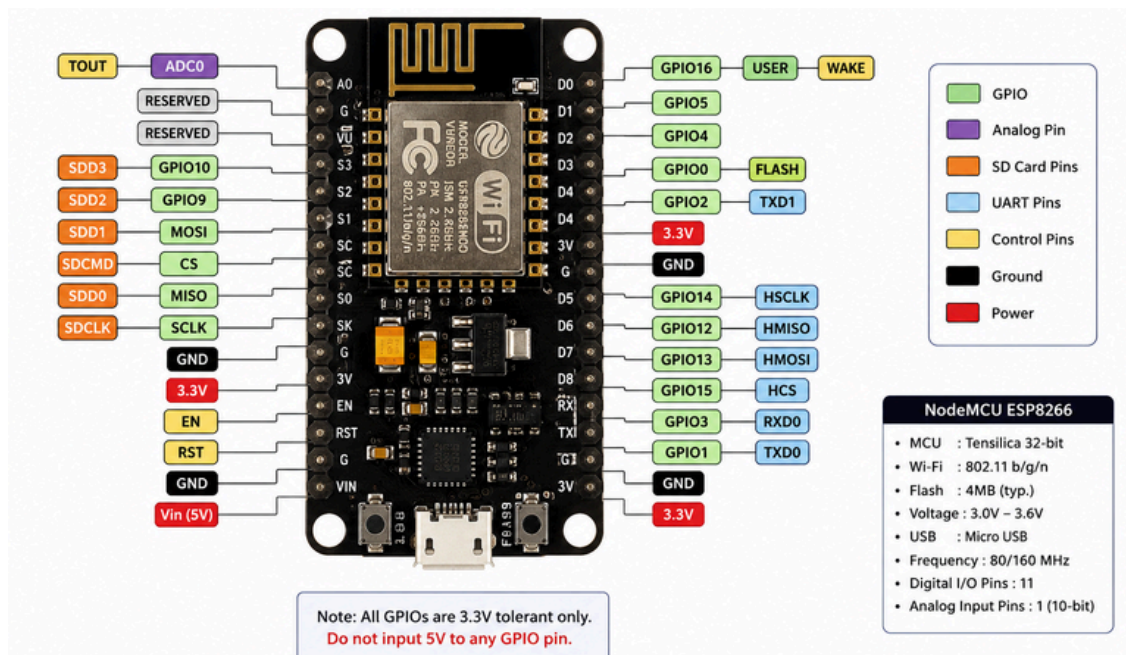
Arduino UNO

- The Arduino UNO serves as the main control unit of the robotic arm, responsible for processing control commands and generating PWM signals to drive the SG90 servo motors. It coordinates the movement of the arm joints by controlling the position of each servo according to the desired motion.



NodeMCU ESP8266

- The NodeMCU (ESP8266) is used in the project to provide wireless communication and IoT connectivity for the robotic arm. It enables the system to connect to a Wi-Fi network and exchange data between the physical robotic arm and the Digital Twin environment.
- The NodeMCU receives control commands from the user interface and transmits them to the Arduino UNO for robotic arm operation. It also supports remote monitoring and control through IoT platforms, allowing users to interact with the robotic arm from any location. Its built-in Wi-Fi capability, compact size, and low cost make it an ideal choice for implementing wireless communication in the project.



SG90 Servo Motors

- The SG90 servo motors are used as the actuators of the robotic arm, providing precise angular movement at each joint. These micro servo motors are lightweight, compact, and capable of rotating approximately 180° , making them suitable for small-scale robotic applications.
- In this project, SG90 servos are responsible for controlling the base, shoulder, elbow, wrist, and gripper movements of the 5-DOF robotic arm. The servos receive PWM (Pulse Width Modulation) signals from the Arduino UNO, allowing accurate positioning of each joint. Due to their low cost, ease of interfacing, and adequate torque for lightweight payloads, SG90 servo motors were selected as an effective solution for the robotic arm system



Power Supply

- A reliable power supply is essential for the proper operation of the robotic arm and its control electronics. In this project, power is provided using two 3.7V lithium-ion batteries connected through a battery holder, ensuring portability and continuous operation.
- The battery system supplies the required energy to the SG90 servo motors, Arduino UNO, and NodeMCU modules. The rechargeable lithium-ion batteries were selected due to their high energy density, lightweight nature, and long operating life. This power arrangement enables stable robotic arm performance while maintaining a compact and portable design.

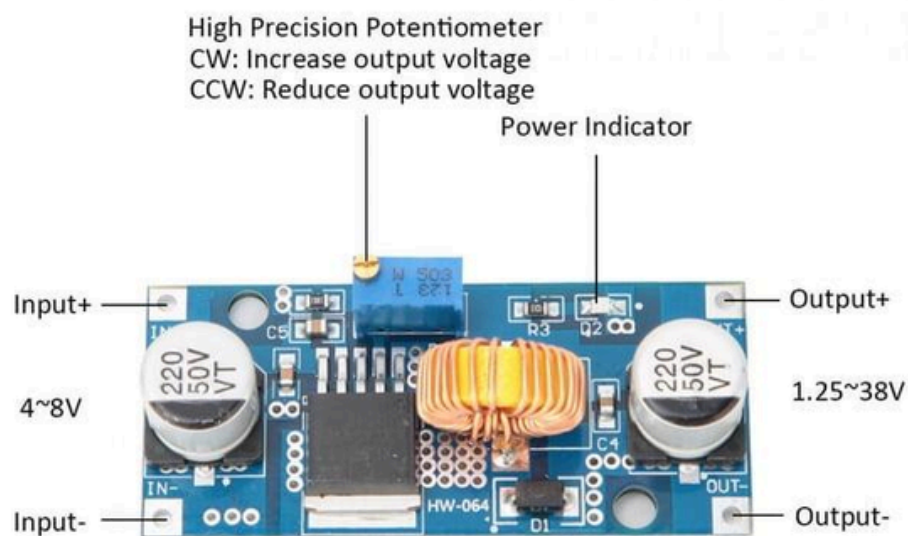
Batteries

- The robotic arm is powered using two 3.7V lithium-ion rechargeable batteries, which serve as the primary energy source for the system. These batteries provide sufficient voltage and current to operate the servo motors and control electronics efficiently.



Buck Converter

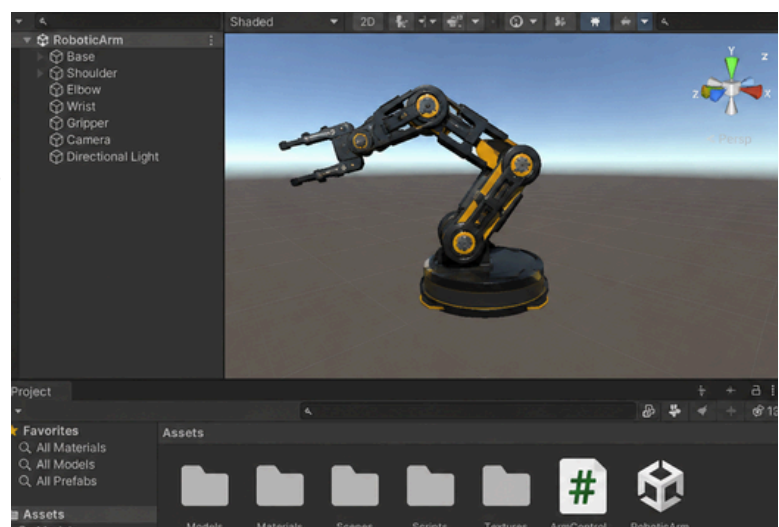
- A buck converter is a DC-DC voltage regulator used to step down a higher input voltage to a lower, stable output voltage. In this project, the buck converter is used to regulate the voltage supplied by the lithium-ion batteries, ensuring that the electronic components receive the required operating voltage safely and efficiently.
- The buck converter helps provide a stable power supply to the Arduino UNO, NodeMCU, and SG90 servo motors, protecting them from voltage fluctuations and improving overall system reliability. Its high efficiency and compact size make it suitable for portable robotic applications.



3.2 Software Tools

• Unity

Unity is a powerful cross-platform development environment used for creating 2D, 3D, virtual reality (VR), and simulation applications. In this project, Unity is used to develop the Digital Twin of the robotic arm by creating a virtual model that replicates the structure and movements of the physical system.



- **Blender**

Blender was used to assemble and visualize the 3D components of the robotic arm in a virtual environment before physical fabrication. The imported STL files of individual parts were positioned, aligned, rotated, and assembled to create the complete robotic arm model. Using Blender helped verify the proper fit and orientation of the components, identify assembly issues, and ensure that the overall structure was correctly formed before integration into the Digital Twin environment. This reduced design errors and provided a clear visualization of the final robotic arm assembly.

- **Arduino IDE**

The Arduino Integrated Development Environment (IDE) is the software platform used for writing, compiling, and uploading programs to the Arduino UNO microcontroller. It provides a user-friendly interface that simplifies the development and testing of embedded applications.

In this project, Arduino IDE was used to develop the control program for the robotic arm, enabling the operation of SG90 servo motors and communication between the Arduino UNO and NodeMCU modules. The software facilitated code development, debugging, and real-time testing, ensuring reliable control of the robotic arm system.

- **Python**

Python plays a crucial role in the development of the intelligent robotic arm system by enabling data generation, processing, and machine learning implementation. It provides a simple and powerful programming environment for performing mathematical calculations and developing predictive models.

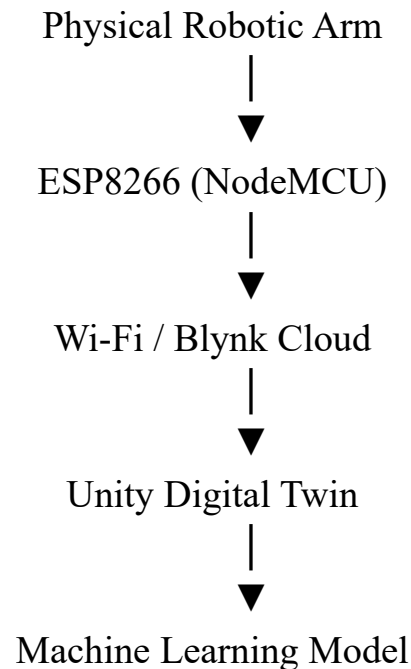
In this project, Python was used to generate the robotic arm dataset using kinematic equations, preprocess the data, and train a machine learning model for joint angle prediction. The trained model helps estimate the required joint angles for a desired end-effector position, improving the intelligence and automation capabilities of the robotic arm.

- **TensorFlow**

TensorFlow is an open-source machine learning framework used for developing and training artificial intelligence models. It provides powerful tools for data processing, neural network development, model training, and prediction.

In this project, TensorFlow was used to train a machine learning model using the robotic arm dataset generated from kinematic calculations. The model learns the relationship between the end-effector coordinates and the corresponding joint angles. After training, the model can predict the required joint angles for a desired position, thereby assisting in the intelligent control of the robotic arm and enhancing the functionality of the Digital Twin system.

3.3 System Architecture



The physical robotic arm serves as the primary hardware component of the system and performs movements based on user commands. The ESP8266 (NodeMCU) acts as a communication interface, enabling wireless data exchange between the robotic arm and external applications through a Wi-Fi network.

The Blynk platform provides a user-friendly interface for remotely controlling and monitoring the robotic arm. The transmitted data is received by the Unity-based Digital Twin, which creates a virtual representation of the physical robotic arm and replicates its movements in real time. This synchronization allows users to visualize and analyze the behavior of the robotic arm without directly interacting with the hardware.

The data generated from the robotic arm's movements and positions is further utilized for machine learning applications. A Machine Learning Model is trained using the generated dataset to learn the relationship between end-effector positions and joint angles. The trained model assists in predicting the required joint angles for a desired position, thereby improving the automation, accuracy, and intelligence of the robotic arm system. The integration of robotics, IoT, Digital Twin technology, and machine learning makes the system suitable for research, education, and industrial automation applications.

Chapter 4: IoT-Based Control of the Robotic Arm Using ESP8266 (NodeMCU)

The Internet of Things (IoT) is a technology that enables physical devices to connect to the internet, allowing them to exchange data and be monitored or controlled remotely. IoT integrates sensors, microcontrollers, communication modules, and cloud platforms to create intelligent systems that can be accessed from anywhere using a smartphone, tablet, or computer.

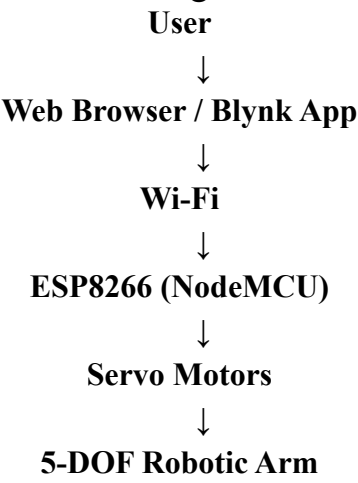
In this project, IoT is implemented using the ESP8266 (NodeMCU) microcontroller, which provides built-in Wi-Fi connectivity. The NodeMCU acts as a bridge between the robotic arm and the internet, enabling users to control the arm wirelessly through a web interface and the Blynk mobile application. Commands sent from the user interface are transmitted over Wi-Fi to the ESP8266, which then controls the servo motors to perform the desired movements.

- **Remote Accessibility:** The robotic arm can be controlled from any location with an internet connection, eliminating the need for direct physical access.
- **Wireless Operation:** IoT removes the dependency on wired communication, making the system more flexible and convenient to operate.
- **Real-Time Control:** Commands are transmitted instantly through Wi-Fi, enabling smooth and responsive movement of the robotic arm.
- **User-Friendly Interface:** A web page and the Blynk mobile application provide an intuitive graphical interface, allowing users to operate the robotic arm without specialized technical knowledge.
- **Scalability:** The IoT architecture can be easily expanded by integrating additional sensors, cameras, or automation features in future developments.
- **Industrial Relevance:** IoT-enabled robotic systems are widely used in modern industries for remote monitoring, smart manufacturing, warehouse automation, and hazardous environment operations

4.2 ESP8266 (NodeMCU)

- **Built-in Wi-Fi:** Enables direct internet connectivity without requiring any external communication modules.
- **Low Cost:** More economical than using an Arduino Uno with a separate Wi-Fi shield or module.
- **Compact Design:** Combines a microcontroller and Wi-Fi interface into a single development board, reducing circuit complexity.
- **High Processing Speed:** Operates at 80 MHz (up to 160 MHz), which is significantly faster than the 16 MHz clock speed of the Arduino Uno.
- **Easy IoT Integration:** Supports web servers, HTTP communication, MQTT, and cloud platforms such as Blynk, making remote control straightforward.

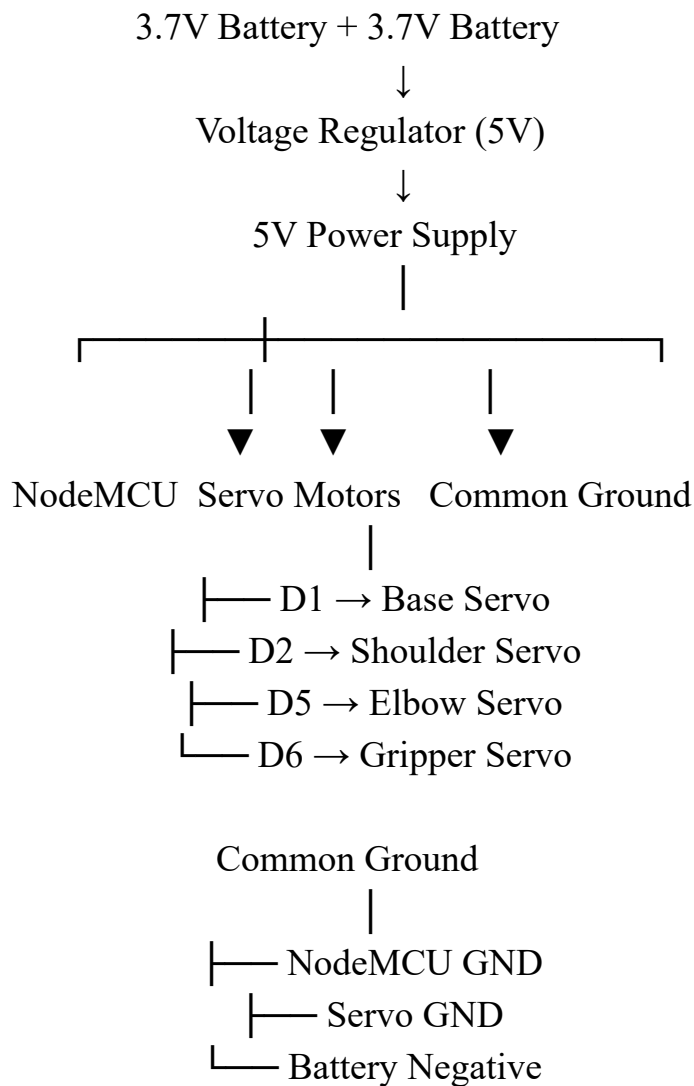
- **Sufficient GPIO Pins:** Provides enough input/output pins to interface with servo motors and other peripherals used in the robotic arm.
- **Large Developer Community:** Extensive documentation and open-source libraries simplify development and troubleshooting.



4.3 Hardware Connections

Component	ESP8266 Pin	Power Connection
Base Servo	D1 (GPIO5)	External +5V
Shoulder Servo	D2 (GPIO4)	External +5V
Elbow Servo	D5 (GPIO14)	External +5V
Gripper Servo	D6 (GPIO12)	External +5V
Servo Grounds	GND	Common Ground
ESP8266	VIN / USB	5V Supply
Battery	Voltage Regulator	5V Output

4.4 Programming the ESP8266



Arduino IDE Setup

1. Download and install the latest version of the Arduino IDE.
2. Launch the Arduino IDE after installation.
3. Connect the ESP8266 (NodeMCU) to the computer using a USB cable.
4. Install the required ESP8266 board package.
5. Select the appropriate board and COM port before uploading the program.

Servo	ESP8266 Pin
Base Servo	D1 (GPIO5)
Shoulder Servo	D2 (GPIO4)
Elbow Servo	D5 (GPIO14)
Gripper Servo	D6 (GPIO12)

Wi-Fi Configuration

The ESP8266 connects to a wireless network using the network SSID and password provided in the program. After establishing the connection, it obtains an IP address from the Wi-Fi router.

Once connected, the ESP8266 can:

- Receive commands from the Blynk mobile application.
- Host a web server for browser-based control.
- Exchange data over the local network or the internet.
- Enable real-time wireless operation of the robotic arm.

The connection status can be monitored through the Arduino IDE's Serial Monitor, where the assigned IP address is displayed after a successful connection. This IP address can then be entered into a web browser to access the robotic arm's control interface.

Web-Based Control

5.5 Creating an Embedded Web Server

One of the key features of this project is the ability to control the robotic arm through a web browser. The ESP8266 (NodeMCU) hosts a lightweight web server that generates a control webpage accessible from any device connected to the same Wi-Fi network. This eliminates the need for additional software installation, as users can simply enter the IP address of the ESP8266 into a web browser to access the control interface.

Working Principle

1. The ESP8266 connects to the Wi-Fi network and starts its web server.
2. The router assigns an IP address to the ESP8266.
3. The user enters this IP address into a web browser.
4. The browser loads the robotic arm control webpage hosted by the ESP8266.
5. When the user presses a control button or adjusts a slider, the browser sends an HTTP request to the ESP8266.
6. The ESP8266 processes the received request.
7. Based on the command, the ESP8266 updates the PWM signal sent to the corresponding servo motor.
8. The servo motor rotates to the specified angle, causing the robotic arm to move.

HTTP Requests

The Hypertext Transfer Protocol (HTTP) is the communication protocol used between the web browser (client) and the ESP8266 (server). Each user action on the webpage generates an HTTP request containing the desired command.

Web Server Initialization

- Creates a web server object.
- Defines the webpage that will be displayed to the user.
- Registers URLs or endpoints to handle different control commands.
- Starts listening for incoming client requests on the HTTP port.

Client–Server Communication

The web-based control system follows the client–server architecture.

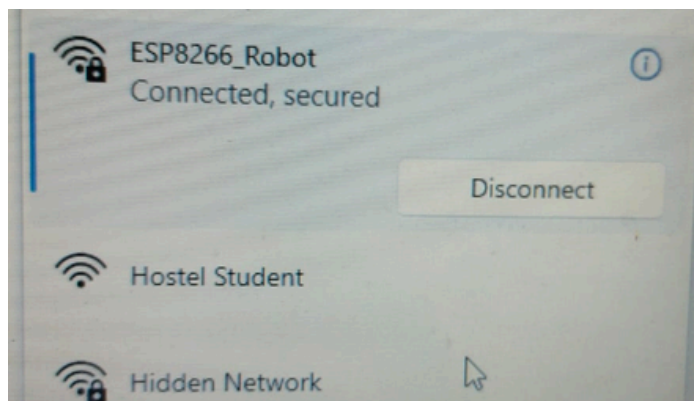
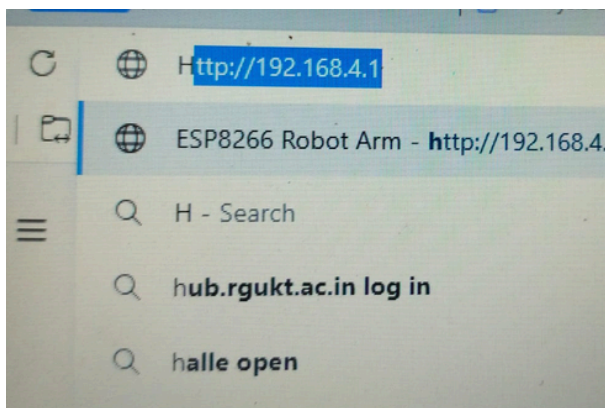
- **Client:** The web browser running on a smartphone, tablet, or computer.
- **Server:** The ESP8266 (NodeMCU), which hosts the control webpage and processes user commands.

The communication process is as follows:

1. The client requests the robotic arm control webpage.
2. The ESP8266 sends the webpage to the browser.
3. The user interacts with the controls on the webpage.
4. The browser sends HTTP requests containing the selected commands.
5. The ESP8266 interprets these commands.
6. PWM signals are generated to drive the appropriate servo motors.
7. The robotic arm performs the requested movement.

4.5 Obtaining the IP Address

- Connecting the ESP8266 to a Wi-Fi network
- Displaying the assigned IP address on the Serial Monitor
- Using the IP address to access the control page



4.6 Designing the Web Page

A web-based user interface (UI) was developed to provide an intuitive and convenient method for controlling the robotic arm remotely. The webpage is hosted directly on the ESP8266 (NodeMCU), allowing users to access it by entering the ESP8266's IP address into any standard web browser connected to the same Wi-Fi network.

User Interface Design

- Base Rotation – Rotates the base of the robotic arm.
- Shoulder Joint – Controls the upward and downward movement of the shoulder.
- Elbow Joint – Controls the bending of the arm.
- Wrist Joint – Adjusts the orientation of the end effector.
- Gripper – Opens and closes the gripper to grasp or release objects.

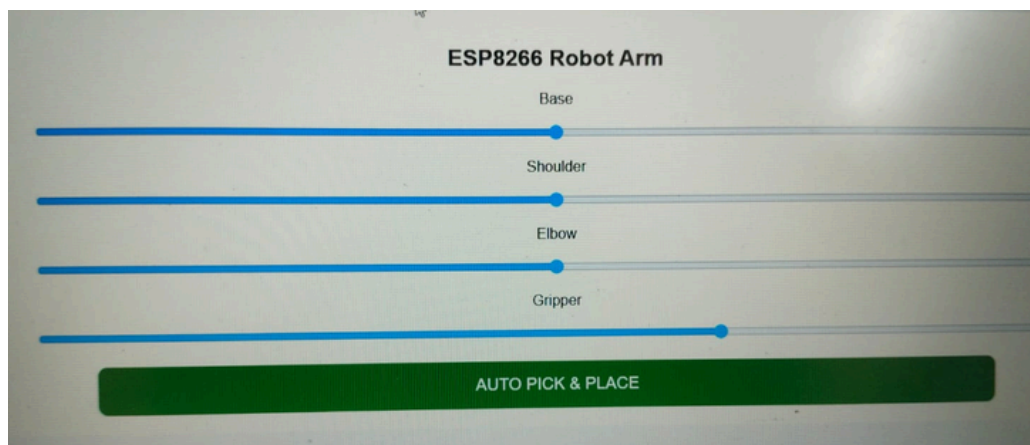
HTML Structure

The HTML page includes:

1. A main title for the robotic arm control system.
2. Labels for each servo motor.
3. Slider controls for selecting servo angles.
4. A section to display the current angle value (optional).
5. JavaScript functions that send commands to the ESP8266 whenever a slider is moved.

Slider Controls

Sliders are used to control the angular position of each servo motor. Each slider typically operates within a range of 0° to 180° , corresponding to the movement limits of the SG90 servo motors.



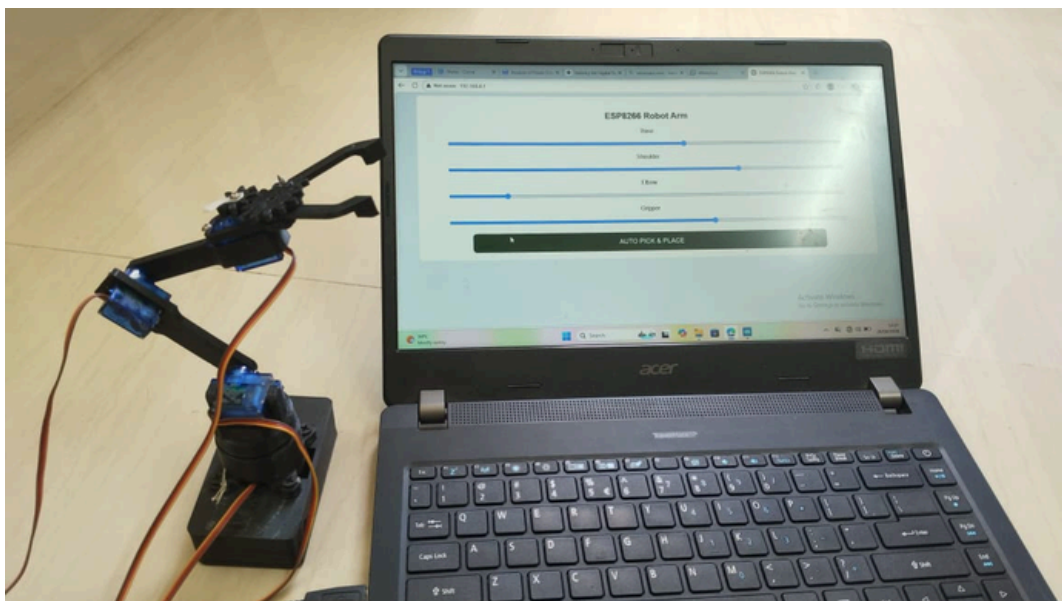
Code:

https://github.com/saimadhav101022/Digital_twin_robotic_arm/tree/main/Web_Based_Control

Control	Servo Motor	Function
Base Rotation	Servo 1	Rotates the base left or right.
Shoulder	Servo 2	Moves the shoulder joint up
Elbow	Servo 3	Controls the bending of the
Wrist	Servo 4	Rotates or tilts the wrist.
Gripper	Servo 5	Opens and closes the gripper.

4.7 Controlling the Robotic Arm Through the Webpage

1. Open browser.
2. Enter the ESP8266 IP address.
3. Webpage loads.
4. User adjusts sliders or presses buttons.
5. HTTP request is sent.
6. ESP8266 processes the request.
7. Servo rotates.
8. Robotic arm moves.



Blynk-Based Control

4.8 Introduction to Blynk

- Blynk is an Internet of Things (IoT) platform that enables users to remotely monitor and control hardware devices through a smartphone application. It provides an easy-to-use interface for developing IoT projects without requiring extensive knowledge of mobile application development. The platform consists of three main components: the Blynk mobile application, the Blynk Cloud server, and the hardware device (ESP8266 NodeMCU).
- In this project, the Blynk platform is used to control the robotic arm wirelessly. The ESP8266 (NodeMCU) connects to the internet via Wi-Fi and communicates with the Blynk Cloud. Commands sent from the Blynk mobile application are received by the ESP8266, which then controls the servo motors to perform the required movements.

Features of Blynk

- **Remote Device Control:** Allows users to control the robotic arm from anywhere with an internet connection.
- **User-Friendly Interface:** Provides drag-and-drop widgets for creating custom control panels without mobile app programming.
- **Real-Time Communication:** Enables instant communication between the smartphone and the ESP8266.
- **Cross-Platform Support:** Available for both Android and iOS devices.
- **Cloud Integration:** Stores project information and manages communication through the Blynk Cloud.
- **Custom Widgets:** Supports buttons, sliders, switches, joysticks, LEDs, displays, and graphs for flexible interface design.
- **Secure Communication:** Uses authentication tokens to ensure that only authorized users can access the device.

Creating a New Template

1. Open the Blynk Console.
2. Click Developer Zone.
3. Select Templates.
4. Click New Template.
5. Enter:

Parameter	Value
Template Name	Robotic Arm
Hardware	ESP8266
Connection Type	Wi-Fi

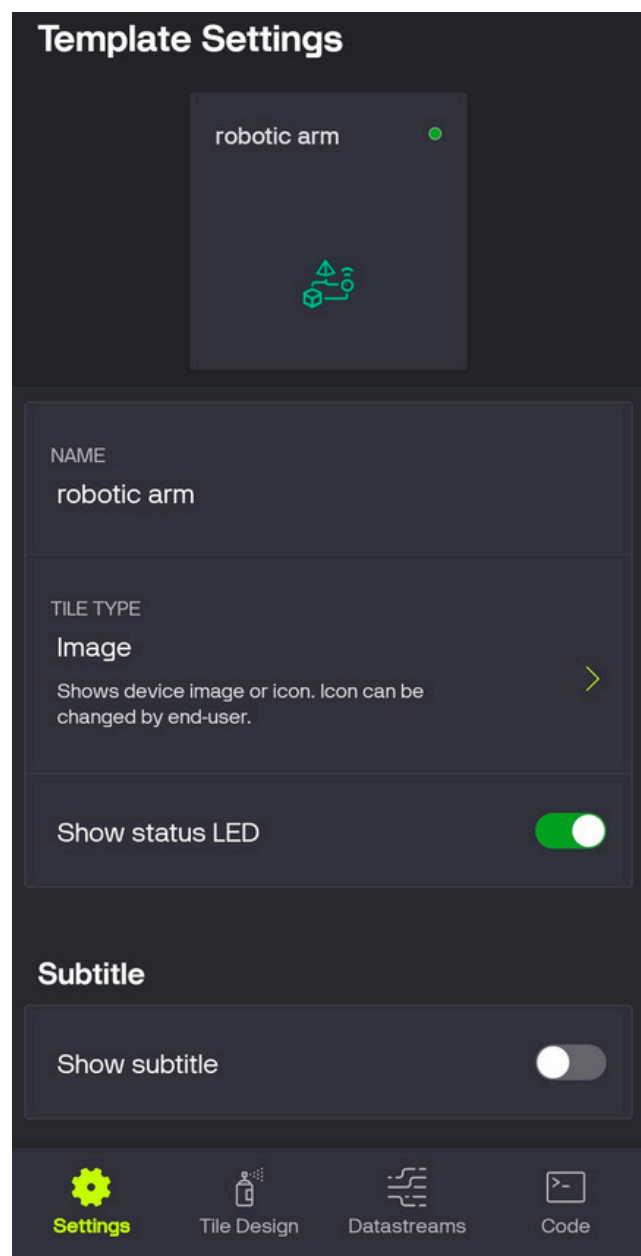
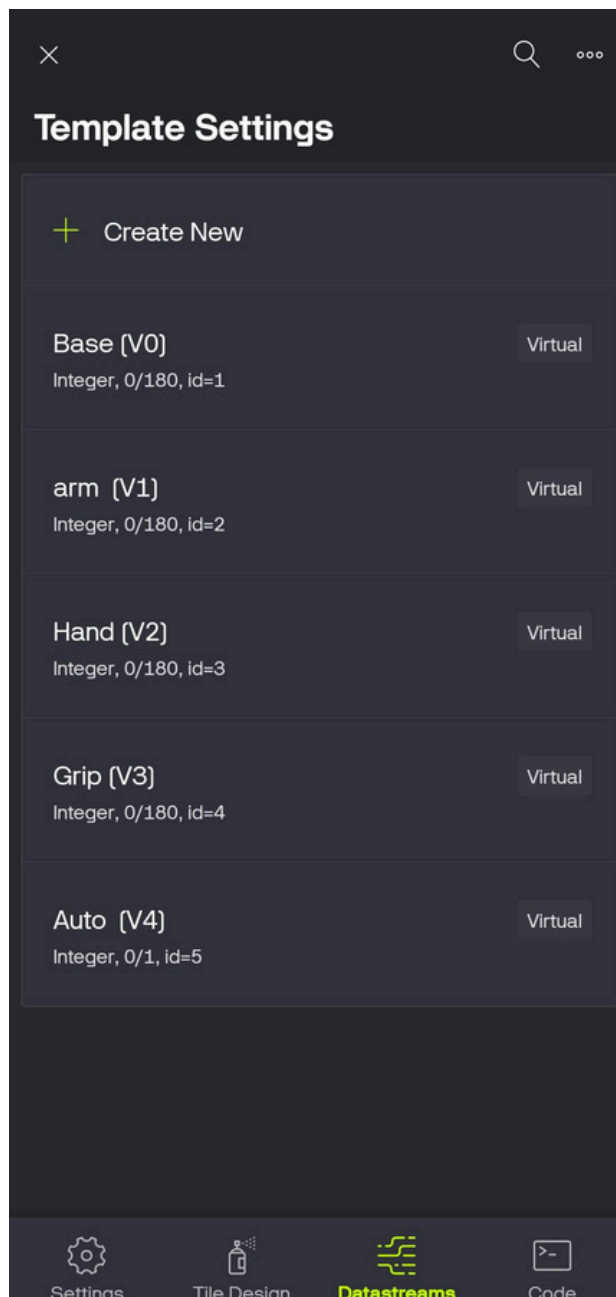
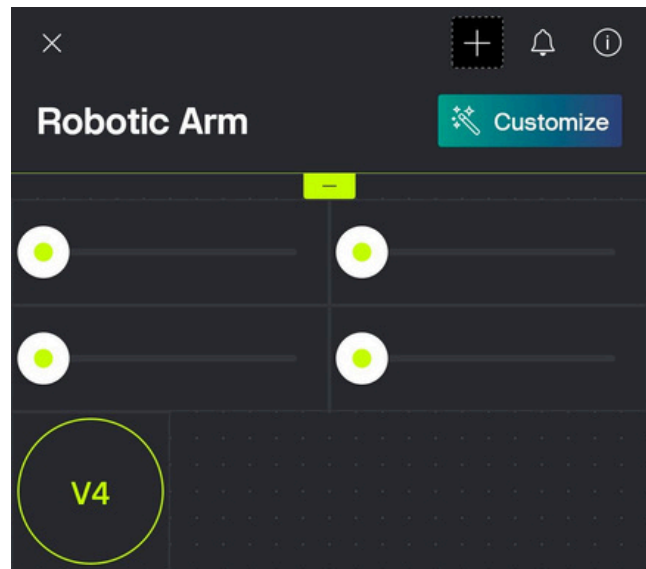
Creating a Device

1. Open Devices.
2. Click New Device.
3. Select From Template.
4. Choose Robotic Arm.
5. Give the device a name.
6. Click Create.

After creation, Blynk generates:

- Template ID
- Device Name
- Authentication Token

These credentials are copied into the Arduino program.



Creating Datastreams

Datastream	Virtual Pin	Data Type	Range
Base	V0	Integer	0–180
Shoulder	V1	Integer	0–180
Elbow	V2	Integer	0–180
Wrist	V3	Integer	0–180
Gripper	V4	Integer	0–180

Creating the Mobile Dashboard

1. Open the Blynk mobile application.
2. Select the created device.
3. Tap Edit Dashboard.
4. Press the + icon.
5. Add Slider widgets.
6. Repeat until five sliders are added.

Configuring Each Slider

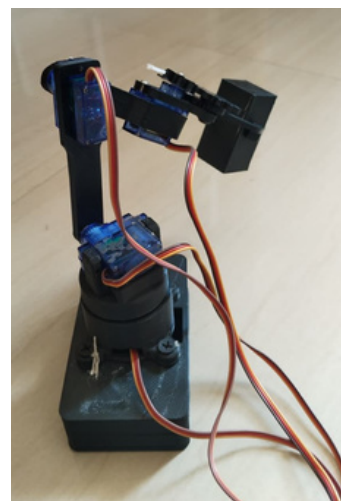
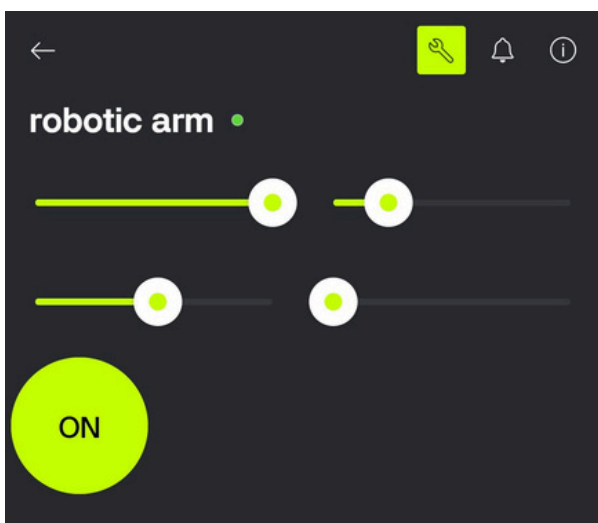
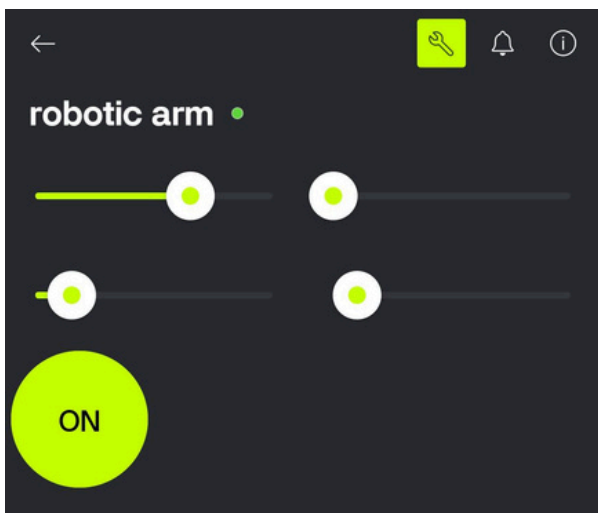
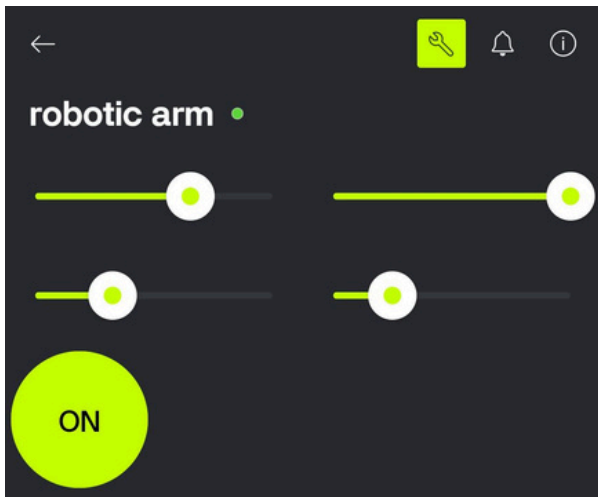
- Assign each slider to the appropriate datastream

Slider	Datastream	Range
Base Rotation	V0	0–180°
Shoulder	V1	0–180°
Elbow	V2	0–180°
Wrist	V3	0–180°
Gripper	V4	0–180°

Connecting ESP8266 to Blynk

In the Arduino program, enter the following details:

- Template ID
- Device Name
- Authentication Token
- Wi-Fi SSID
- Wi-Fi Password



Controlling the Robotic Arm

1. Once the ESP8266 is online:
2. Open the Blynk application.
3. Select the robotic arm dashboard.
4. Move any slider.
5. The slider value is sent to the Blynk Cloud.
6. The cloud forwards the value to the ESP8266.
7. The ESP8266 reads the corresponding virtual pin.
8. The servo motor rotates to the selected angle.
9. The robotic arm moves accordingly

Comparison Between Web-Based Control and Blynk Control

Feature	Web Page	Blynk App
Interface	Browser	Mobile App
Internet Access	Local Wi-Fi (or configured)	Internet via Blynk Cloud
Ease of Use	Moderate	High
Customization	HTML/CSS/JavaScript	Drag-and-drop widgets
Response Time	Fast on local network	Depends on internet

Code:

[Digital_twin_robotic_arm/Blynk_IoT_Control at main · saimadhav101022/Digital_twin_robotic_arm](#)

Chapter 5: Dataset Generation, Machine Learning Training, Testing, and Prediction

Machine Learning (ML) was incorporated into the robotic arm system to predict the appropriate joint angles for a desired end-effector position. Instead of manually calculating inverse kinematics every time, a neural network model was trained using a dataset generated through forward kinematics. The complete process consists of dataset generation, data preprocessing, model training, performance evaluation, and prediction.

Step 1: Dataset Generation Using Python

The first step is to generate a large dataset that represents different configurations of the robotic arm. Since collecting thousands of real-world measurements is time-consuming, a Python program was developed to generate synthetic data using forward kinematics equations.

The program performs the following operations:

- Defines the robotic arm link lengths.
- Generates random joint angles.
- Converts angles from degrees to radians.
- Computes the end-effector coordinates (X, Y, Z).
- Calculates the effective arm length.
- Estimates the payload capacity.
- Stores all generated values in a CSV file.
- The generated dataset contains 100,000 samples, which are sufficient for training the machine learning model.

Dataset generation is one of the most important stages in the development of the intelligent robotic arm because it provides the data required to train and evaluate the machine learning model. In this project, the dataset consists of robotic arm joint angles, end-effector positions (X, Y, and Z coordinates), and torque values generated using forward kinematics. A large number of samples were created to represent different arm configurations and movements within the robot's workspace.

The generated dataset enables the machine learning model to learn the relationship between the end-effector position and the corresponding joint angles. As a result, the trained model can accurately predict the required servo motor angles for a desired target position, reducing computational complexity compared to solving inverse kinematics repeatedly.

A well-generated dataset improves the accuracy, reliability, and generalization capability of the machine learning model. It forms the foundation of the digital twin by enabling intelligent prediction, real-time simulation, and synchronization between the physical robotic arm and its virtual counterpart. Therefore, dataset generation plays a crucial role in achieving the objectives of Industry 4.0 by integrating robotics, artificial intelligence, and digital twin technology into a single intelligent system.

Code:

```
import numpy as np
import pandas as pd

# Number of samples
num_samples = 100000

# Link lengths (meters)
L1 = 0.02
L2 = 0.062
L3 = 0.063
L4 = 0.067

data = []

for _ in range(num_samples):

    # Random joint angles
    J1 = np.random.uniform(0, 180)
    J2 = np.random.uniform(20, 160)
    J3 = np.random.uniform(20, 160)
    J4 = np.random.uniform(0, 180)

    # Convert to radians
    t1 = np.radians(J1)
    t2 = np.radians(J2)
    t3 = np.radians(J3)
    t4 = np.radians(J4)

    # Effective reach
    effective_length = (
        L2*np.cos(t2) +
        L3*np.cos(t2+t3) +
        L4*np.cos(t2+t3+t4)
    )

    effective_length = abs(effective_length)

    # End effector position
    X = effective_length*np.cos(t1)
    Y = effective_length*np.sin(t1)

    Z = (
        L1 +
        L2*np.sin(t2) +
        L3*np.sin(t2+t3) +
        L4*np.sin(t2+t3+t4)
    )
```

```

        # Payload estimation
        payload = 70 * (0.08 / (effective_length + 0.02))

        payload = max(5, min(payload, 70))

        data.append([
            J1,
            J2,
            J3,
            J4,
            X,
            Y,
            Z,
            effective_length,
            payload
        ])

        columns = [
            "J1",
            "J2",
            "J3",
            "J4",
            "X",
            "Y",
            "Z",
            "Effective_Length",
            "Payload_g"
        ]

        df = pd.DataFrame(data, columns=columns)

        df.to_csv("robotic_arm_payload_dataset.csv", index=False)

        print(df.head())
        print("\nDataset saved as robotic_arm_payload_dataset.csv")

```

Explanation

- Import Required Libraries
- Define Number of Samples
- Define Link Lengths
- Create an Empty List
- Generate Random Joint Angles
- Convert Degrees to Radians
- Calculate Effective Reach
- Convert Effective Reach to Positive Value

- Calculate End-Effector Position
- Estimate Payload
- Limit the Payload Range
- Store One Dataset Sample
- Define Dataset Columns
- Create a DataFrame
- Save the Dataset
- Display the Dataset
- Display Confirmation Message

Convert Degrees to Radians

$$\theta_{\text{rad}} = \theta_{\text{deg}} \times 180 / \pi$$

Calculate Effective Reach

$$L_{\text{eff}} = L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3) + L_4 \cos(\theta_2 + \theta_3 + \theta_4)$$

Calculate End-Effector Position

- **X-coordinate**

The X-coordinate depends on Base rotation (J1) ,Effective reach

$$X = L_{\text{eff}} \cos \theta_1$$

- **Y-coordinate**

$$Y = L_{\text{eff}} \sin \theta_1$$

- **Z-coordinate**

The Z-coordinate represents the vertical height of the end effector.

$$Z = L_1 + L_2 \sin \theta_2 + L_3 \sin(\theta_2 + \theta_3) + L_4 \sin(\theta_2 + \theta_3 + \theta_4)$$

Estimate Payload

$$\text{payload} = 70 * (0.08 / (\text{effective_length} + 0.02))$$

This equation estimates the payload based on the arm's reach.

- When the arm is closer to the base (smaller effective length), it can support a larger payload.
- As the arm extends farther, the payload decreases because the torque on the joints increases.

This is an engineering approximation used to generate training data rather than a direct physical measurement.

Dataset Structure

Feature	Description
J1	Base Angle
J2	Shoulder Angle
J3	Elbow Angle
J4	Wrist Angle
X	End-Effector X Position
Y	End-Effector Y Position
Z	End-Effector Z Position
Effective Length	Current Reach
Payload_g	Estimated Payload

Data Preprocessing

Before training, the dataset is preprocessed.

The preprocessing steps include

- Loading the CSV dataset.
- Separating input and output variables.
- Scaling the data using StandardScaler.
- Splitting the dataset into training and testing sets.

Scaling improves convergence and helps the neural network learn more efficiently.

• Data_Set URI

[Digital_twin_robotic_arm/Data_set at main · saimadhav101022/Digital_twin_robotic_arm](#)

Step 2: Training the Neural Network

A TensorFlow neural network is used to learn the relationship between the robot's position and the corresponding joint angles.

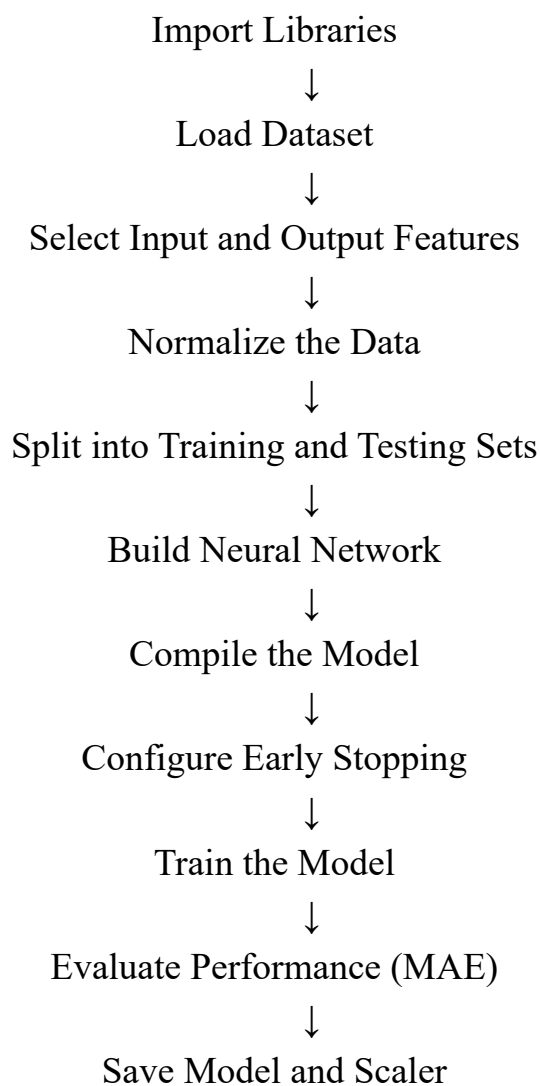
The training process includes:

During each epoch:

- Input data is provided.
- Predictions are generated.
- Error is calculated.
- Network weights are updated.

The prediction accuracy gradually improves.

```
Epoch 57/100      1s 3ms/step - loss: 0.0286 - mae: 0.1340 - val_loss: 0.0265 - val_mae: 0.1273
250/250
Epoch 58/100      1s 3ms/step - loss: 0.0329 - mae: 0.1448 - val_loss: 0.0150 - val_mae: 0.1273
250/250
Epoch 59/100      1s 3ms/step - loss: 0.0314 - mae: 0.1404 - val_loss: 0.0431 - val_mae: 0.1273
250/250
Epoch 60/100      1s 3ms/step - loss: 0.0271 - mae: 0.1304 - val_loss: 0.0135 - val_mae: 0.1273
250/250
Epoch 61/100      1s 3ms/step - loss: 0.0318 - mae: 0.1387 - val_loss: 0.1273 - val_mae: 0.1273
250/250
Epoch 62/100      1s 3ms/step - loss: 0.0311 - mae: 0.1377 - val_loss: 0.0168 - val_mae: 0.1273
250/250
Epoch 63/100      1s 3ms/step - loss: 0.0350 - mae: 0.1492 - val_loss: 0.0650 - val_mae: 0.1273
250/250
Epoch 64/100      1s 3ms/step - loss: 0.0259 - mae: 0.1270 - val_loss: 0.0256 - val_mae: 0.1273
250/250
Epoch 65/100      1s 3ms/step - loss: 0.0250 - mae: 0.1242 - val_loss: 0.0232 - val_mae: 0.1273
250/250
Epoch 66/100      1s 3ms/step - loss: 0.0260 - mae: 0.1273 - val_loss: 0.0164 - val_mae: 0.1273
250/250
```



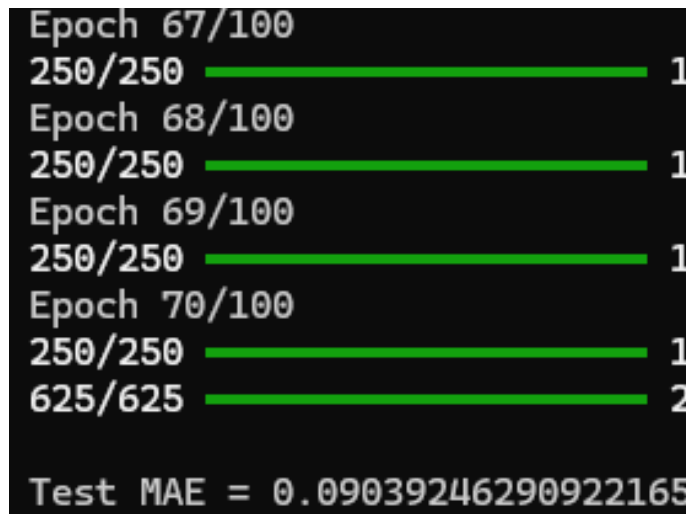
Model Evaluation

After training, the model performance is evaluated using testing data.

The following metrics are monitored:

- Training Loss
- Validation Loss
- Mean Absolute Error (MAE)

A low MAE indicates that the predicted values are close to the actual values.



```
Epoch 67/100
250/250 ██████████ 1
Epoch 68/100
250/250 ██████████ 1
Epoch 69/100
250/250 ██████████ 1
Epoch 70/100
250/250 ██████████ 1
625/625 ██████████ 2
Test MAE = 0.09039246290922165
```

Step 5: Prediction

Once the neural network has been trained, it can predict the robotic arm's joint angles for a desired end-effector position.

Prediction process:

1. Enter X, Y and Z coordinates.
2. Scale the input.
3. Feed the data into the trained neural network.
4. Obtain predicted joint angles.
5. Convert predictions into servo commands.
6. Move the robotic arm to the desired position.

Thus, the trained model enables automatic motion planning without manually solving inverse kinematics.

- **Machine Learning Source Code**

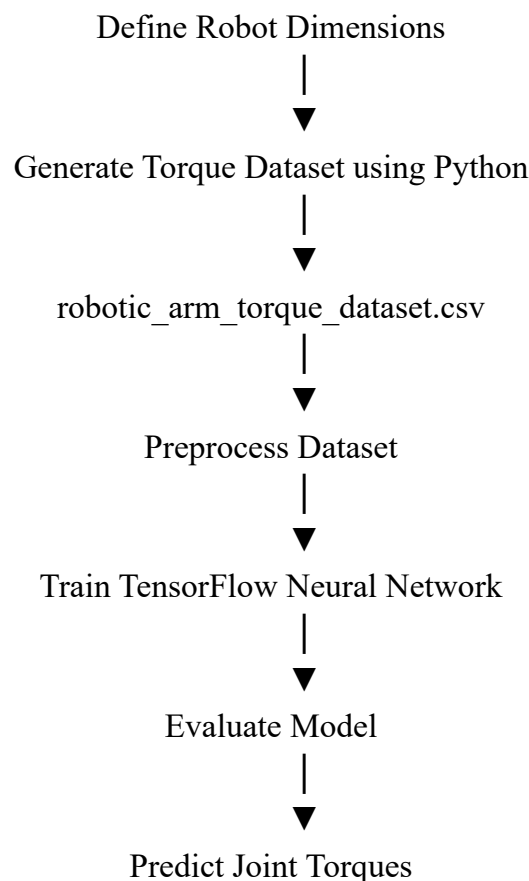
[Digital_twin_robotic_arm/Machine_learning_at_main :](https://github.com/saimadhav101022/Digital_twin_robotic_arm)
[saimadhav101022/Digital_twin_robotic_arm](https://github.com/saimadhav101022/Digital_twin_robotic_arm)

```
ate computer flags.  
C:\Users\madhav\AppData\Local\Programs  
re names, but StandardScaler was fitte  
warnings.warn(  
WARNING:tensorflow:TensorFlow GPU supp  
be used. Please use WSL2 or the Tensor  
1/1 ————— 0s 94ms/step  
Predicted Payload = 48.099403 grams
```

Torque Estimation Using Machine Learning

Torque estimation is an important aspect of robotic arm design because it determines the amount of rotational force required by each servo motor to lift a given payload. Instead of manually calculating torque for every robotic arm configuration, a machine learning model was developed to predict the torque required at each joint. The complete process includes dataset generation, preprocessing, model training, evaluation, and prediction.

Overall Workflow



Step 1: Torque Dataset Generation Using Python

A Python program was developed to automatically generate a dataset containing different robotic arm configurations. Instead of collecting experimental measurements, torque values were calculated mathematically using the robot dimensions, payload, and gravitational force. The generated dataset was then used to train the machine learning model.

Step-by-Step Explanation of the Code

Step 1: Import Required Libraries

The program imports:

NumPy – mathematical and trigonometric calculations.

Pandas – creating and saving the dataset

Step 2: Define Robot Parameters

- Number of samples = 100000
- Link lengths:
 - $L1 = 0.02$ m
 - $L2 = 0.062$ m
 - $L3 = 0.063$ m
 - $L4 = 0.067$ m
- Gravitational acceleration ($g = 9.81$ m/s²)

Step 3: Generate Random Joint Angles

Random values are generated for:

- J1 (Base)
- J2 (Shoulder)
- J3 (Elbow)
- J4 (Wrist)

Random payload values between 5 g and 70 g are also generated

Step 4: Convert Payload

The payload is converted from grams to kilograms before torque calculations because SI units are required.

Step 5: Calculate Forward Kinematics

- - Effective reach
 - X coordinate, Y coordinate, Z coordinate

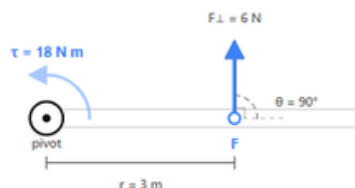
Step 6: Calculate Joint Torques

The torque required at each joint is calculated from the payload and link lengths.

$$\tau = rF \sin(\theta)$$
$$\tau = (3)(6) \sin(90^\circ) = 18 \text{ N m}$$

Only the perpendicular part of F turns the lever.

F	6.0	N	
r	3.0	m	
θ	90.0	deg	



Using the robot geometry:

- J4 Torque = Payload $\times$ Gravity $\times$ L4
- J3 Torque = Payload $\times$ Gravity $\times$ (L3 + L4)
- J2 Torque = Payload $\times$ Gravity $\times$ (L2 + L3 + L4)
- J1 Torque = 0 Nm (base rotation is not modeled for payload torque in this dataset).

Dataset Structure

Input Features	Output Targets
J1	Torque J2
J2	Torque J3
J3	Torque J4
J4	
X	
Y	
Z	
Payload	

Data Preprocessing

Before training:

- Load the CSV dataset.
- Separate inputs and outputs.
- Normalize the input data using StandardScaler.
- Save the scaler.
- Split the dataset into training (80%) and testing (20%).

Robotic Arm Dataset

[Digital_twin_robotic_arm/Data_set](#) at main · saimadhav101022/Digital_twin_robotic_arm

Step 2: Neural Network Training

A TensorFlow neural network is created with multiple fully connected (Dense) layers.

The model:

- Accepts 8 input features.
- Learns the relationship between robot configuration and torque.
- Predicts three outputs:
- Shoulder torque
- Elbow torque
- Wrist torque

The model uses:

- **Optimizer:** Adam
- **Loss Function:** Mean Squared Error (MSE)
- **Evaluation Metric:** Mean Absolute Error (MAE)

Early Stopping is applied to prevent overfitting by stopping training when the validation loss no longer improves.

```
Epoch 97/100
250/250 — 1s 2ms/step - loss: 4.4291e-08 - mae: 1.6146e-04 - val_loss: 1.3808e-08 - val_mae: 9.0919e-05
Epoch 98/100
250/250 — 1s 2ms/step - loss: 3.9913e-08 - mae: 1.5047e-04 - val_loss: 1.6795e-07 - val_mae: 3.7518e-04
Epoch 99/100
250/250 — 1s 2ms/step - loss: 3.5699e-08 - mae: 1.4082e-04 - val_loss: 1.3067e-08 - val_mae: 9.4884e-05
Epoch 100/100
250/250 — 1s 2ms/step - loss: 3.0514e-08 - mae: 1.3090e-04 - val_loss: 1.2240e-08 - val_mae: 7.9776e-05
625/625 — 1s 899us/step - loss: 8.8641e-09 - mae: 7.1657e-05
Test MAE = 7.165696297306567e-05
```

Model Evaluation

- Test data is used to evaluate performance.
- The model reports:
- Test Loss
- Mean Absolute Error (MAE)

A lower MAE indicates that the predicted torque values are closer to the actual torque values.

Step 3: Torque Prediction

After the model is trained, it is used to predict torque values for new robotic arm configurations.

The prediction process is:

1. Enter:

- Joint angles (J1–J4)
- X, Y, Z coordinates
- Payload

2. Scale the input using the saved scaler.

3. Feed the data into the trained neural network.

4. The model predicts:

- Shoulder torque (J2)
- Elbow torque (J3)
- Wrist torque (J4)

These predicted values help determine whether the selected servo motors can safely handle the required load.

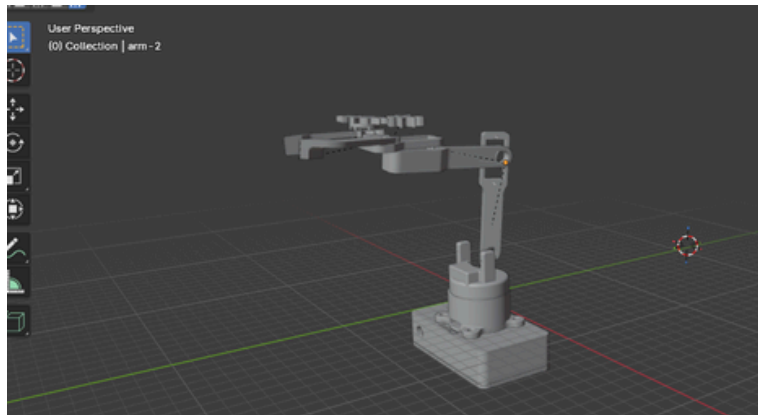
```
Enter J1: 27
Enter J2: 45
Enter J3: 32
Enter J4: 28
Enter X (m): 2
Enter Y (m): 2
Enter Z (m): 2
Enter Payload (g): 30
C:\Users\madhav\AppData\Local\Programs\Python\Python39\Scripts\python.exe: warning:
re names, but StandardScaler was fitted with
warnings.warn(

===== RESULTS =====
Torque J2 (Shoulder) = -0.055889763 Nm
Torque J3 (Elbow)    = 0.16895644 Nm
Torque J4 (Wrist)    = 0.2679451 Nm
```


6.Digital Twin Development and VR Integration

Step 1: Assemble the 3D Robotic Arm in Blender

All individual STL files of the robotic arm components, including the base, shoulder, elbow, wrist, and gripper, were imported into Blender. The components were aligned and assembled according to the actual physical dimensions to create a complete 3D model of the robotic arm.



Step 2: Export the 3D Model

After completing the assembly of all robotic arm components in Blender, the entire model was exported in the FBX (Filmbox) format. The FBX format was chosen because it preserves the 3D geometry, object hierarchy, pivot points, transformations, and material information during the export process. The exported FBX file was then imported into Unity, where it served as the digital twin of the physical robotic arm. This ensured that the model maintained its structural integrity and allowed individual joints to be controlled accurately during simulation and real-time operation.

- To achieve realistic movement, individual joints were linked according to the kinematic structure of the robotic arm. C# scripts were developed and attached to the corresponding joints, allowing each servo motor angle received from the ESP8266 to control the rotation of its digital counterpart. This enabled real-time synchronization between the physical robotic arm and its virtual model.
- The successful import of the 3D model into Unity laid the foundation for developing the digital twin. It provided an interactive virtual environment where robotic arm movements could be visualized, monitored, and tested in real time without affecting the physical hardware. This process significantly improved system analysis, debugging, and demonstration, making the digital twin an effective tool for smart manufacturing and Industry 4.0 applications.

Blender and 3D Models

[Digital_twin_robotic_arm/Blender at main · saimadhav101022/Digital_twin_robotic_arm](#)

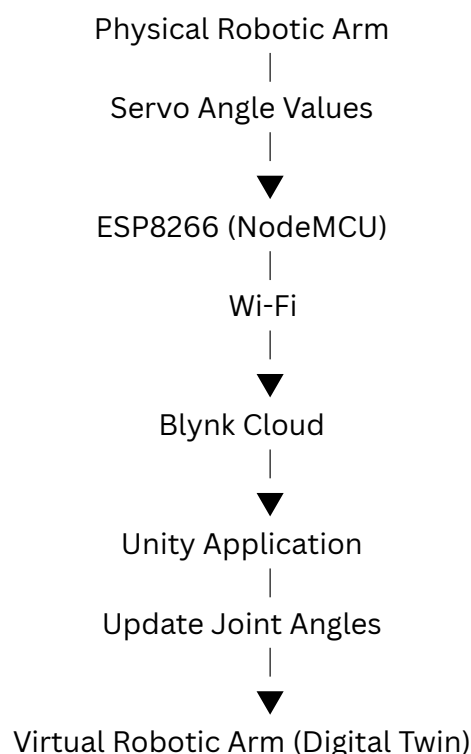
Step 3: Import the Model into Unity

The exported FBX model was imported into Unity. Each robotic arm link was organized, and pivot points were adjusted to ensure accurate joint rotations. The virtual model was then prepared to function as the digital twin of the physical robotic arm.



Step 4: Synchronize with the Physical Robotic Arm

After importing the robotic arm model into Unity, the virtual model was synchronized with the physical robotic arm using the ESP8266 (NodeMCU) and the Blynk IoT platform. The NodeMCU continuously transmitted the real-time servo motor angles over Wi-Fi through Blynk. Unity received these angle values and applied them to the corresponding joints of the virtual robotic arm. As a result, every movement performed by the physical robotic arm was simultaneously reflected in the Unity environment, creating a real-time digital twin. This synchronization enabled accurate visualization, monitoring, and control of the robotic arm within the virtual environment.



Step 5: Verify Joint Movements

After establishing synchronization between the physical robotic arm and its digital twin, the movement of each joint was verified to ensure accurate operation. Each servo motor was actuated individually through the Blynk mobile application, and the corresponding joint in the Unity model was observed. The movements of the base, shoulder, elbow, wrist, and gripper were checked for correct direction, rotation range, and responsiveness. Any discrepancies in joint orientation or angle mapping were corrected within Unity to achieve accurate synchronization. Successful verification confirmed that the virtual robotic arm precisely mirrored the real-time movements of the physical robotic arm, demonstrating the effectiveness of the digital twin system.

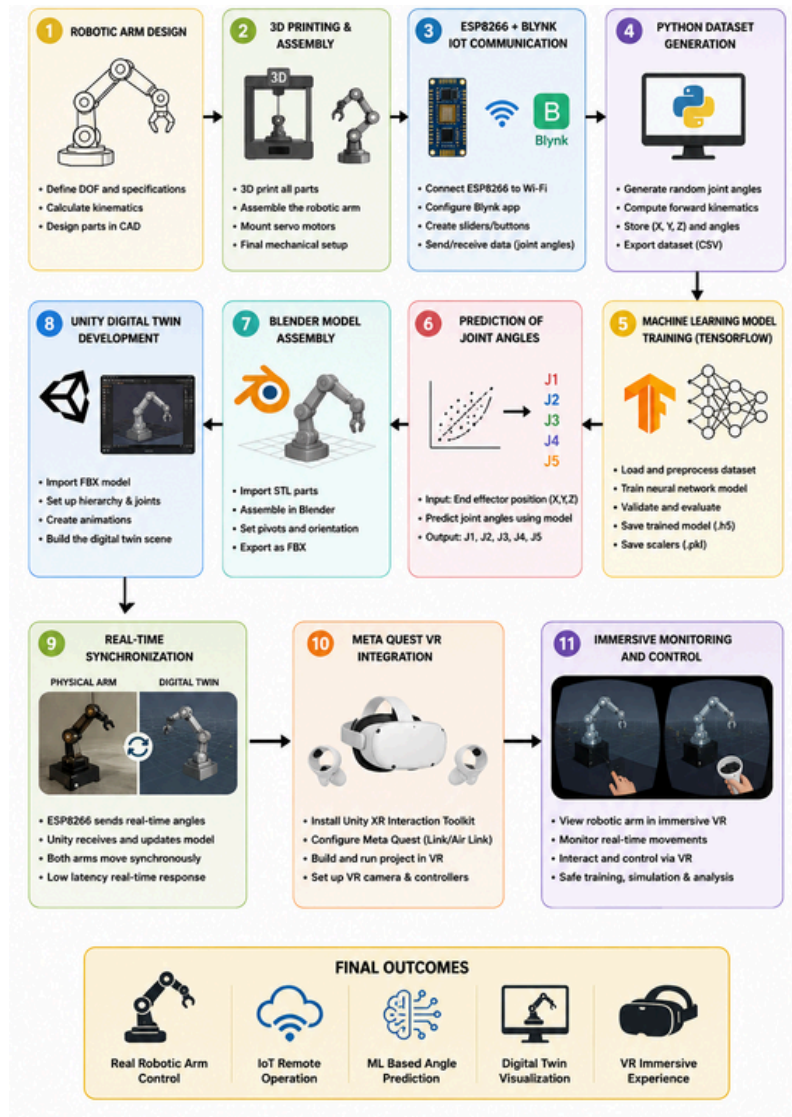


Verification Procedure

1. Move each servo motor individually using the Blynk mobile application.
2. Observe the corresponding joint movement in the Unity digital twin.
3. Verify the rotation direction and angular range of each joint.
4. Adjust joint orientation or angle mapping if necessary.
5. Repeat the process for all joints until the virtual and physical robotic arms move synchronously.
6. Confirm smooth, accurate, and real-time synchronization between the physical robotic arm and its digital twin.

This step ensures that the digital twin accurately represents the physical robotic arm, providing reliable visualization and real-time monitoring during operation.

VR Integration



VR Integration Using Meta Quest

Step 1: Meta Quest Device Setup

The Meta Quest headset was selected as the Virtual Reality device for this project because of its standalone operation, high-resolution display, and compatibility with the Unity game engine. The headset was configured using the Meta Quest application and connected to the development computer through a USB cable and wireless network. Developer Mode was enabled to allow testing and deployment of VR applications directly from Unity. The device firmware was updated to ensure compatibility with the latest Unity XR tools. Proper calibration of the headset and controllers was performed before executing the VR application to obtain accurate tracking and comfortable visualization.

Step 2: Installation of Meta XR Plugin

The Meta XR All-in-One SDK was imported into the Unity project to enable communication between Unity and the Meta Quest headset. This SDK provides essential components such as camera tracking, controller input, hand tracking, and interaction systems required for Virtual Reality applications. The OpenXR plugin was enabled in Unity's Project Settings to establish compatibility with the Meta Quest hardware. After importing the required packages, all XR dependencies were verified to ensure smooth execution of the VR environment.



Step 3: XR Origin Configuration

An XR Origin was added to the Unity scene to represent the user's position within the virtual environment. The XR Origin includes the main VR camera and controller objects responsible for tracking the user's head and hand movements. The camera automatically updates its position based on the user's physical movements, creating an immersive first-person experience. The XR Origin was positioned to provide a clear view of the robotic arm while maintaining a comfortable interaction distance. This configuration allows users to freely observe the robotic arm from different angles inside the virtual environment.

Step 4: Meta Quest Controller Integration

The Meta Quest controllers were configured using Unity's XR Interaction Toolkit. Controller buttons, triggers, thumbsticks, and grip controls were mapped to various interaction functions within the virtual environment. These controllers allow users to navigate around the robotic arm, inspect individual joints, and interact with virtual objects. Smooth controller tracking provides a natural user experience and enables future extensions such as virtual robot manipulation and interactive control panels.

Step 5: Digital Twin Synchronization

The digital twin developed in Unity was integrated with the Meta Quest environment. The ESP8266 transmitted real-time servo motor angles through Wi-Fi, and Unity continuously updated the corresponding joints of the virtual robotic arm. The Meta Quest headset displayed these synchronized movements in a fully immersive three-dimensional environment. This enabled users to monitor robotic operations while feeling physically present inside the virtual workspace. The synchronization process accurately represented the behavior of the physical robotic arm within the digital twin.

Step 6: Immersive Visualization

The Meta Quest headset provides stereoscopic 3D visualization, allowing users to observe the robotic arm with realistic depth perception. Users can walk around the robotic arm, inspect the mechanical structure from different viewpoints, and closely examine the movement of each joint. The immersive environment improves understanding of robotic kinematics, workspace limitations, and motion planning. This visualization is particularly beneficial for engineering education, virtual demonstrations, and industrial operator training.



Step 7: Real-Time Monitoring and Interaction

During operation, the Meta Quest headset continuously displayed the live status of the robotic arm. Any movement performed by the physical robotic arm was immediately reflected in the virtual environment through the digital twin. Users could monitor robotic motion, identify abnormal behavior, and evaluate system performance without approaching the physical hardware. The combination of IoT communication and Virtual Reality significantly enhances monitoring efficiency and operational safety.

Step 8: Testing and Performance Evaluation

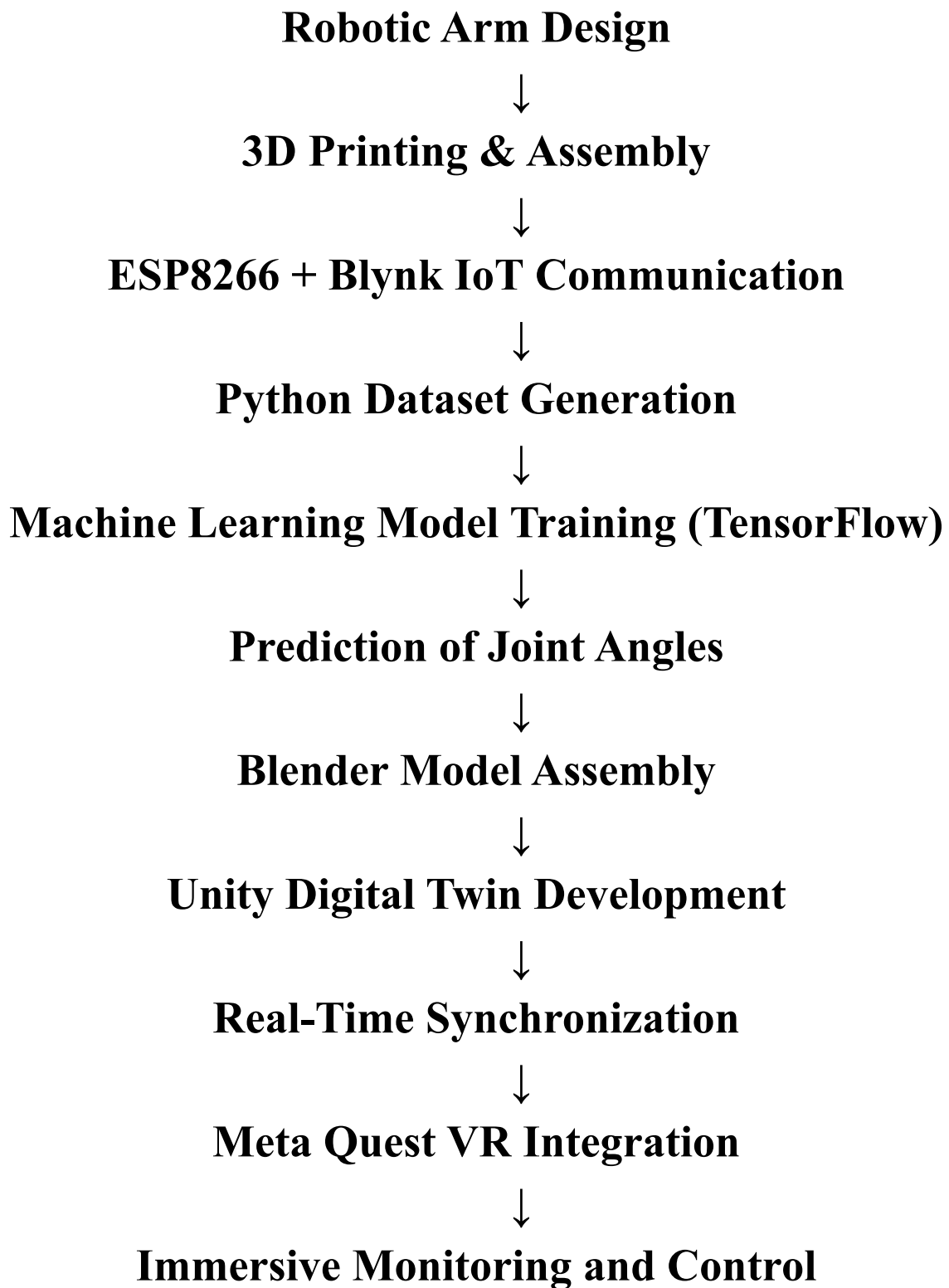
Several experimental tests were conducted to evaluate the performance of the VR-integrated digital twin. Different robotic arm movements, including pick-and-place operations, were executed while monitoring synchronization between the physical and virtual systems. The Meta Quest headset successfully displayed smooth robotic motion with minimal communication delay. The results confirmed accurate tracking, stable rendering, and reliable synchronization, demonstrating the effectiveness of the proposed Industry 4.0-based Digital Twin system.

Step 9: Final Implementation

The successful integration of Meta Quest with the Unity-based Digital Twin resulted in a fully immersive robotic visualization platform. The completed system combines IoT, Unity simulation, Machine Learning, and Virtual Reality into a unified Industry 4.0 application. The developed platform enables real-time monitoring, virtual inspection, operator training, and intelligent robotic visualization while reducing operational risks and improving user understanding of robotic systems. The project demonstrates the practical implementation of modern Digital Twin technology for future smart manufacturing applications.



Overall Workflow



CHAPTER 7: Applications

The developed Digital Twin-based robotic arm has a wide range of applications across industrial automation, education, research, and smart manufacturing. The integration of IoT, Machine Learning, and Virtual Reality makes the system suitable for real-time monitoring, remote operation, and intelligent decision-making.

7.1 Industrial Automation

The robotic arm can perform repetitive operations such as pick-and-place, sorting, assembly, and material handling in manufacturing industries. The Digital Twin enables operators to monitor the system remotely and detect abnormalities before actual execution.

7.2 Remote Robot Operation

Using the ESP8266 Wi-Fi module and the Blynk platform, the robotic arm can be controlled remotely from anywhere with an internet connection. This feature is useful in hazardous environments where direct human intervention is unsafe.

7.3 Virtual Reality Training

The integration of the Meta Quest VR headset allows users to interact with the virtual robotic arm in an immersive environment. Operators can practice robot handling without risking damage to physical equipment.

7.4 Educational Laboratories

The developed system serves as an effective educational platform for learning robotics, IoT, machine learning, digital twins, and industrial automation. Students can understand robot kinematics and control concepts through real-time visualization.

7.5 Predictive Maintenance

Machine learning can be extended to monitor robotic performance and identify abnormal operating conditions. This helps schedule maintenance before failures occur, reducing downtime and maintenance costs.

7.6 Smart Manufacturing (Industry 4.0)

The project demonstrates Industry 4.0 concepts by integrating physical hardware, cloud communication, digital twins, and intelligent algorithms into a unified smart manufacturing system.

7.7 Human–Robot Interaction

The Digital Twin and VR environment improve visualization and interaction between humans and robots, enabling safer collaboration and easier monitoring of robotic tasks.

Conclusion

- This project successfully developed a Digital Twin-based 5-DOF robotic arm capable of real-time synchronization between the physical robot and its virtual counterpart. The robotic arm was designed using CAD models, fabricated through 3D printing, assembled, and controlled using servo motors interfaced with an ESP8266-based IoT system.
- A virtual replica of the robotic arm was created in Unity using Blender-generated 3D models. Real-time communication between the physical robot and the Digital Twin was achieved through Wi-Fi using the Blynk platform, enabling remote monitoring and control.
- Machine Learning techniques were implemented using TensorFlow to predict robotic joint movements from the generated dataset, improving the intelligence of the system and demonstrating the potential for data-driven robotic control. The integration of the Meta Quest VR headset further enhanced user interaction by providing an immersive virtual training environment.
- The project successfully combines Robotics, IoT, Machine Learning, Digital Twin technology, and Virtual Reality into a single Industry 4.0 framework. The developed system offers a cost-effective, scalable, and educational platform suitable for industrial automation, remote operation, operator training, and future research. Overall, the project demonstrates the practical implementation of modern smart manufacturing concepts and highlights the significant role of Digital Twin technology in the next generation of intelligent robotic systems.

References

- [1] "Digital Twins: A Survey on Enabling Technologies, Challenges, Trends and Future Prospects," IEEE Communications Surveys & Tutorials, 2022.
- [2] IEEE, "IEEE Xplore Digital Library," Available: <https://ieeexplore.ieee.org>
- [3] Espressif Systems, ESP8266 Technical Reference Manual. Available: <https://www.espressif.com>
- [4] Blynk, Blynk IoT Platform Documentation. Available: <https://docs.blynk.io>
- [5] TensorFlow, TensorFlow Documentation. Available: <https://www.tensorflow.org>
- [6] Unity, Unity User Manual. Available: <https://docs.unity3d.com>